

`hdc_stream_wrapper.sv` — AXI4-Stream Wrapper and Co-Simulation Flow

This note documents the high-throughput streaming interface around the verified end-to-end inference core (`hdc_core_top`), and the co-simulation that streams every window through it under randomized handshake timing. For the protocol itself, see the companion `AXI4_Stream_Protocol_Study.pdf`.

1. Purpose

The AXI4-Lite wrapper is the *control* path: fine for configuration and one-off inferences, but each window costs several CPU-driven register writes plus status polling. For continuous biosignal classification the PS should instead fill a memory buffer with quantized windows and let an **AXI-DMA** engine stream them into the PL — zero CPU work per window. `hdc_stream_wrapper` is the PL endpoint of that path:

```
PS (DDR buffer) -> AXI-DMA (MM2S) -> S_AXIS  hdc_stream_wrapper  M_AXIS -> AXI-DMA
(S2MM) -> PS
                                     (hdc_core_top inside)
```

2. Interface

Port group	Signals	Meaning
Clock/reset	<code>aclk</code> , <code>aresetn</code>	single clock, active-low reset
Config	<code>proto_we/proto_idx/proto_vec</code> , <code>mask_we/mask_vec</code>	prototype + pruning-mask write ports (passed straight to <code>popcount_am</code>); driven by the AXI-Lite staging logic in a full system
S_AXIS	<code>s_axis_tvalid/tready/tdata[31:0]/tlast</code>	windows in — 3 beats per window, <code>TLAST</code> on the final beat
M_AXIS	<code>m_axis_tvalid/tready/tdata[31:0]/tlast</code>	results out — one beat per window: <code>(class_idx << 16) \ </code> <code>class_dist</code> , <code>TLAST=1</code>

Window framing (input)

The packed level grid is 80 bits (20 pairs × 4-bit level), little-endian over 3 beats: beat *b* carries `levels_flat[b*32 +: 32]`; the upper 16 bits of beat 2 are ignored. The result encoding matches the AXI-Lite RESULT register, so software parses both paths identically.

3. Internals

A three-state FSM around the verified core — no datapath logic of its own:

- **ST_IN** — `s_axis_tready` high; each accepted beat lands at its word offset in `levels_reg`. `TLAST` (or the beat counter as a safety net) commits the window and pulses `core.start`.
- **ST_RUN** — wait for `core.out_valid` (~24 clocks: encode 20 pairs + classify), then drive the result beat with `m_axis_tvalid/tlast`.
- **ST_OUT** — hold the beat until `m_axis_tready`; back to ST_IN.

Back-pressure is correct on both sides: input ready is dropped while an inference is in flight, and the result beat is held stable until consumed — so the wrapper can sit behind a stalled DMA without losing data.

Throughput at the default configuration: 3 input beats + ~24 core cycles + 1 output beat ≈ **28 clocks per window** when the stream is not throttled (≈ 3.5 M windows/s at 100 MHz — far beyond any biosignal rate; the design is latency-trivial and leaves headroom for larger D or more classes).

4. Co-simulation

Testbench — `tb/tb_stream_cosim.sv`

Reuses the deployment-style golden from `generate_vectors.py --core` (trained prototypes, ~75 % pruning mask, level grids, expected labels — the item-memory `.mem` files are shared with the DUT's ROMs). After loading prototypes + mask once through the config ports, each case runs **two concurrent processes**:

- `send_window` streams the 3 beats with a **random 0–3 cycle idle gap** before each beat;
- `recv_result` waits for a true `TVALID && TREADY` edge and checks `class_idx`, `class_dist`, and `TLAST` against the golden.

Meanwhile `m_axis_tready` is **randomized every cycle** (~60 % duty), forcing the wrapper to hold result beats under back-pressure. This randomized timing is what distinguishes a protocol test from a datapath test: a handshake bug (sampling on `TVALID` alone, dropping a held beat) cannot hide.

Harness — `sim/run_stream_cosim.do`

```
vsim -c -do sim/run_stream_cosim.do      # NUM_CASES env var (default 200)
```

Regenerates the golden → fresh `work` → compiles the full core + wrapper + TB → streams all windows.

5. Result

```
tb_stream_cosim: AXI4-Stream + hdc_core_top vs golden
VECDIR = python_ref/vectors/cosim_core
CASES   = 200   (D=1024, 3 beats/window, N_CLASS=8)
PASS: all 200 streamed inferences match the Python golden.
```

200 / 200 under randomized valid gaps and back-pressure on ModelSim SE-64 10.6e.

6. Where this leaves the project

Both PS-facing interfaces of the HDC core are now verified:

Interface	Wrapper	Co-sim	Use
AXI4-Lite	<code>hdc_core_axi_lite</code>	200/200 PASS	configuration + one-off inference
AXI4-Stream	<code>hdc_stream_wrapper</code>	200/200 PASS	DMA-fed continuous classification

All RTL work for the streaming HDC classifier is complete. What remains is platform bring-up (Vivado block design with PS + AXI-DMA + both wrappers, on-board smoke test), the throughput/latency/energy/area measurements, and the Python novelty studies (Hook A Pareto, Twists 1/2).